

Deep Learning

Assignment 5

Learn Decision trees for regression and classification problem

- Split the data set into training and test sets.
- Build the decision tree
- Check model performances on training and test data sets.
- Apply cost complexity pruning to overcome overfitting problem
- Apply Random Forest algorithm to overcome overfitting problem.
- Apply Ada-boost ensemble method on Decision stumps.

Name: Rohit Kalaskar **Roll No:** 24

PRN: 12310106

Branch: CS AI - B

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import LabelEncoder
```

```
df = pd.read_csv("house_price.csv")
df.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	pr
0	13300000	7420	4	2	3	yes	no	no	no	yes	2	
1	12250000	8960	4	4	4	yes	no	no	no	yes	3	
2	12250000	9960	3	2	2	yes	no	yes	no	no	2	
3	12215000	7500	4	2	2	yes	no	yes	no	yes	3	
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df_encoded = df.copy()
le = LabelEncoder()

for col in df_encoded.columns:
    if df_encoded[col].dtype == 'object':
        df_encoded[col] = le.fit_transform(df_encoded[col])
```

```
X = df_encoded.drop('price', axis=1)
y = df_encoded['price']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```

```
dt_reg = DecisionTreeRegressor(random_state=42)
dt_reg.fit(X_train, y_train)
```

DecisionTreeRegressor ⓘ ?
DecisionTreeRegressor(random_state=42)

```
train_pred = dt_reg.predict(X_train)
test_pred = dt_reg.predict(X_test)

print("Training RMSE:", np.sqrt(mean_squared_error(y_train, train_pred)))
print("Testing RMSE:", np.sqrt(mean_squared_error(y_test, test_pred)))
print("R² Score:", r2_score(y_test, test_pred))
```

Training RMSE: 50263.13114939395
Testing RMSE: 1762815.1944129835
R² Score: 0.3256617355420972

```
path = dt_reg.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas

dt_pruned = DecisionTreeRegressor(
    random_state=42,
    ccp_alpha=ccp_alphas[10]
)

dt_pruned.fit(X_train, y_train)
pruned_pred = dt_pruned.predict(X_test)

print("Pruned RMSE:", np.sqrt(mean_squared_error(y_test, pruned_pred)))
```

Pruned RMSE: 1762169.4970421428

```
rf_reg = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

rf_reg.fit(X_train, y_train)
rf_pred = rf_reg.predict(X_test)

print("Random Forest RMSE:", np.sqrt(mean_squared_error(y_test, rf_pred)))
print("Random Forest R²:", r2_score(y_test, rf_pred))
```

Random Forest RMSE: 1401705.2024909118
Random Forest R²: 0.5736388706262081

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
```

```
df_spam = pd.read_csv("spam.csv", encoding='latin-1')
df_spam = df_spam[['v1', 'v2']]
df_spam.columns = ['label', 'message']
df_spam.head()
```

	label	message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Next steps: [Generate code with df_spam](#) [New interactive sheet](#)

```
df_spam['label'] = df_spam['label'].map({'ham': 0, 'spam': 1})
```

```
X = df_spam['message']
y = df_spam['label']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

vectorizer = CountVectorizer(stop_words='english')
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
```

```
dt_cls = DecisionTreeClassifier(random_state=42)
dt_cls.fit(X_train_vec, y_train)
```

DecisionTreeClassifier ⓘ ?

```
DecisionTreeClassifier(random_state=42)
```

```
train_pred = dt_cls.predict(X_train_vec)
test_pred = dt_cls.predict(X_test_vec)

print("Training Accuracy:", accuracy_score(y_train, train_pred))
print("Testing Accuracy:", accuracy_score(y_test, test_pred))

print(classification_report(y_test, test_pred))
```

```
Training Accuracy: 1.0
Testing Accuracy: 0.9612347451543432
```

	precision	recall	f1-score	support
0	0.97	0.98	0.98	1202
1	0.89	0.82	0.85	191
accuracy			0.96	1393
macro avg	0.93	0.90	0.92	1393
weighted avg	0.96	0.96	0.96	1393

```
path = dt_cls.cost_complexity_pruning_path(X_train_vec, y_train)
ccp_alphas = path.ccp_alphas

dt_cls_pruned = DecisionTreeClassifier(
    random_state=42,
    ccp_alpha=ccp_alphas[5]
)

dt_cls_pruned.fit(X_train_vec, y_train)
pruned_pred = dt_cls_pruned.predict(X_test_vec)

print("Pruned Accuracy:", accuracy_score(y_test, pruned_pred))
```

```
Pruned Accuracy: 0.9619526202440776
```

```
rf_cls = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_cls.fit(X_train_vec, y_train)
rf_pred = rf_cls.predict(X_test_vec)

print("Random Forest Accuracy:", accuracy_score(y_test, rf_pred))
```

```
Random Forest Accuracy: 0.9755922469490309
```

```
ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=50,
    learning_rate=1,
    random_state=42
)
```

```
ada.fit(X_train_vec, y_train)
ada_pred = ada.predict(X_test_vec)
```

```
print("AdaBoost Accuracy:", accuracy_score(y_test, ada_pred))
```

```
AdaBoost Accuracy: 0.9038047379755922
```